Large-Scale Systems Security (SSLE)
Guide 1
(24/09/2026)

Service Oriented Architectures (SOA)

1. Introduction

Service-Oriented Architecture (SOA) is a software architecture model that organizes system functionality into independent, interoperable services. These services are designed to be reusable, modular, and connected through well-defined interfaces, often using communication protocols such as HTTP or AMQP and standards such as REST. The main purpose of SOA is to promote flexibility and agility, allowing different components of a system, or even different systems, to collaborate without being directly coupled. This makes it easier to integrate different applications, adapt to new business needs, scale them, and implement security controls.

2. Microservices Fundamentals

Let us create a basic SOA architecture that we can build upon to explore large-scale security concepts in the following lessons.

We will have a:

- Service Registry
- Ingestion Service
- Temperature Sensor
- Storage Service
- Analytics service

Think about how the service identifiers should allow a service to be:

- discovered (via a registry or DNS),
- addressed (so other services can talk to it), and
- managed (for monitoring, scaling, and load balancing).

Consider how naming should reflect different location levels (e.g., node in a rack, rack in a datacenter, region, country), creating a hierarchical ID that lets you quickly identify the element.

2.1 Service Registry

A Service Registry is like a phonebook or directory where all services register themselves so other services can discover them.

Instead of hardcoding service locations (like http://127.0.0.1:5001), a registry lets services find each other dynamically.

Let us create a service registry that allows for:

Registration

When a service (e.g., IngestionService) starts, it registers its name, IP address, and port with the registry.

Discovery

Other services (e.g., TemperatureSensor) query the registry to find the current location of the InjestionService.

Health Checking

The registry can periodically check whether a service is still alive. If not, it removes it from the directory.

Load Balancing

If multiple instances of the same service exist, the registry can provide different endpoints to distribute traffic.

Take the following code as an example:

```
from flask import Flask, request, jsonify
from datetime import datetime

app = Flask(_name_)
registry = {}

@app.route('/register', methods=['POST'])
def register():
  data = request.get_json()
  name, url = data.get("name"), data.get("url")
  if not name or not url:
    return jsonify({"error": "Missing name or url"}), 400
  registry[name] = {"url": url, "timestamp": datetime.utcnow().isoformat()}
  return jsonify({"message": f"{name} registered"}), 200

@app.route('/unregister', methods=['DELETE'])
def unregister():
  name = request.get_json().get("name")
  if name in registry:
    del registry[name]
    return jsonify({"message": f"{name} unregistered"}), 200
  return jsonify({"error": "Not found"}), 404

@app.route('/services', methods=['GET'])
def services():
  return jsonify(registry)

@app.route('/services/<name>', methods=['GET'])
def get_service(name):
  service = registry.get(name)
  return jsonify(service) if service else (jsonify({"error": "Not found"}), 404)

if_name_== '_main_':
  app.run(port=5050)
```

2.2 Ingestion Service

An ingestion service is a software component that collects incoming data from various sources and moves it into a system where it can be stored, processed, or analyzed.

Think of it as the entry point or funnel for data in an architecture.

Data Collection

Accepts data from different producers (IoT devices, apps, APIs, logs, etc.).

Normalization / Transformation (optional)

Cleans or converts raw data into a consistent format before passing it along.

Routing

Sends the data to the right downstream system (databases, analytics platforms, queues, etc.).

Buffering

Handles bursts of incoming data using queues or streams (e.g., Kafka, RabbitMQ, MQTT).

Take the following code as an example:

```
from flask import Flask, request, jsonify
import requests

app = Flask(_name_)
SERVICE_NAME = "ingestion-service"
SERVICE_URL = "http://localhost:5001"

@app.route('/ingest', methods=['POST'])
def ingest():
  data = request.get_json()
  print("Received:", data)
  # Forward to storage
  try:
    res = requests.post("http://localhost:5002/store", json=data)
    return jsonify({"stored": True}), res.status_code
  except Exception as e:
    return jsonify({"error": str(e)}), 500

def register():
  requests.post("http://localhost:5050/register", json={
    "name": SERVICE_NAME,
    "url": SERVICE_URL
  })
```

```
if_name_== '_main_':
  register()
  app.run(port=5001)
```

2.3 Temperature (Celsius)

A temperature sensor is a device that measures the degree of hotness or coldness of an environment, object, or substance, and converts that physical property into a signal (analog or digital) that a computer or microcontroller can read.

In IoT systems, the temperature sensor is often the data source that kicks off the entire pipeline.

```
import requests
import time
import random

DEVICE_ID = "sensor-001"

def send_temp():
  while True:
    temp = round(random.uniform(20.0, 30.0), 2)
    payload = {"device_id": DEVICE_ID, "temperature": temp}
    try:
      requests.post("http://localhost:5001/ingest", json=payload)
      print("Sent:", payload)
    except Exception as e:
      print("Error:", e)
    time.sleep(5)

if_name_== '_main_':
  send_temp()
```

2.4 Storage Service

A storage service is a dedicated software component or microservice that is responsible for persistently storing, retrieving, and managing data for other services.

Instead of each service managing its database directly, a storage service abstracts the storage layer and exposes APIs for data access.

Data Persistence

Safely write data into a database, filesystem, or object store.

Data Retrieval

Provide APIs to query or fetch stored information.

Abstraction

Hide database complexity from other services. (They don't need to know if you're using MySQL, MongoDB, or S3.)

Consistency & Integrity

Ensure data is not corrupted and supports transactions when necessary.

Access Control

Manage who can read/write data (authentication & authorization).

Take the following code as an example:

```
from flask import Flask, request, jsonify
import sqlite3

app = Flask(_name_)
DB = 'temp_data.db'

def init_db():
  conn = sqlite3.connect(DB)
  c = conn.cursor()
  c.execute('''
    CREATE TABLE IF NOT EXISTS temperatures (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      device_id TEXT,
      temperature REAL,
      timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
    )
  ''')
  conn.commit()
  conn.close()

@app.route('/store', methods=['POST'])
def store():
  data = request.get_json()
  conn = sqlite3.connect(DB)
  c = conn.cursor()
  c.execute("INSERT INTO temperatures (device_id, temperature) VALUES (?, ?)",
        (data["device_id"], data["temperature"]))
  conn.commit()
  conn.close()
  return jsonify({"status": "stored"}), 200

if_name_== '_main_':
  init_db()
  app.run(port=5002)
```

2.5 Analytics Service

An Analytics Service is a specialized service in a system architecture that processes, analyzes, and interprets data to extract insights.

Unlike a storage service (which just saves and retrieves data), an analytics service adds value by turning raw data into meaningful information, such as trends, summaries, predictions, or alerts.

Data Consumption

Pulls data from a **Storage Service** or **Ingestion Service**.

Processing & Aggregation

Performs calculations like averages, min/max values, and trends over time.

Insights Generation

Produces useful outputs like “average temperature in the last hour” or “anomaly detected.”

Exposing Results

Provides APIs (REST/gRPC) so other services (e.g., dashboards, alert systems) can consume insights.

Optional ML/AI Integration

Runs predictive analytics (e.g., forecasting future sensor values).

Take the following code as an example:

```
from flask import Flask, jsonify
import sqlite3

app = Flask(_name_)
DB ='temp_data.db'

@app.route('/average', methods=['GET'])
def average_temp():
  conn = sqlite3.connect(DB)
  c = conn.cursor()
  c.execute("SELECT AVG(temperature) FROM temperatures")
  avg = c.fetchone()[0]
  conn.close()
  return jsonify({"average_temperature": round(avg, 2) if avg else None})

if_name_== '_main_':
  app.run(port=5003)
```

Future work

In this lesson, we have introduced SOA mostly from a functional perspective. Now, we must identify the security requirements for such a system and how to introduce these elements in a way that scales easily.

1. Reflect upon the lack of authentication and authorization. What approaches would you propose?
2. How could we make the system more resilient to function failures and reduce exposure if a single service is compromised?
3. How would you integrate existing cybersecurity tools within such an architecture?